

YAKWETU · NAIROBI

B2C Consumer Conversion & Direct-Purchase Re-Engagement Engine

An automated n8n + AI workflow system that captures high-intent viewers, removes checkout friction, and re-engages abandoners with personalized nudges across WhatsApp, SMS, and email.

Solution design & prototype build

Scope: browse-abandonment recovery · payment-failure rescue · post-watch upsell

Stack: n8n · OpenAI · Postgres · Twilio WhatsApp · Africa's Talking SMS · M-Pesa (simulated)

September 2026

1. Executive Summary

Yakwetu sells African movies directly — no subscription — so every abandoned checkout is permanently lost revenue, not a delayed renewal. This document specifies a working prototype of a **conversion and re-engagement engine**: four n8n workflows, a Postgres state layer, AI-personalized messaging, and a dummy storefront that simulates the full consumer journey (browse → checkout → M-Pesa payment → watch → upsell) so the system can be demonstrated end-to-end without touching production.

The engine covers all three target scenarios:

Table 1 Scenario coverage

Scenario	Trigger	Response	Workflow
A — Browse / cart abandonment	Session idle 45 min+ with no purchase	AI-personalized WhatsApp nudge with one-click resume link; SMS fallback; max 2 nudges	YKW-02
B — Payment failure / checkout drop-off	payment_failed event	Instant failure-class-specific troubleshooting WhatsApp; 2 h later an incentive SMS if unrecovered; drop-off logged	YKW-03
C — Post-watch upsell	watch_completed event	Genre-matched discounted bundle email after credits; WhatsApp last-call 24 h later if unpurchased	YKW-04

Every nudge is logged with its scenario and channel, and every recovered purchase is attributed back to the nudge that caused it — so the demo ends with a measurable recovery-rate number, not just messages on a phone.

2. Design Principles

Single event surface. The storefront (and later, the real app) knows exactly one URL: `POST /webhook/yakwetu-event`. All routing, state, and scenario logic lives inside n8n — the product team never touches the app to change a nudge.

State before messaging. No message is ever sent from a trigger directly. Events first update a session-state table; scheduled and follow-up logic reads that state. This makes quiet hours, frequency caps, and recovery checks trivially reliable.

AI writes, rules decide. The LLM only generates copy and picks recommendations inside strict constraints (length, tone, catalog). Timing, targeting, channel choice, and caps are deterministic rules — auditable and safe.

Graceful channel cascade. WhatsApp first (highest engagement in Kenya), Africa's Talking SMS on failure, email for rich content. Every send is wrapped so a channel outage degrades the message, never the workflow.

3. System Architecture

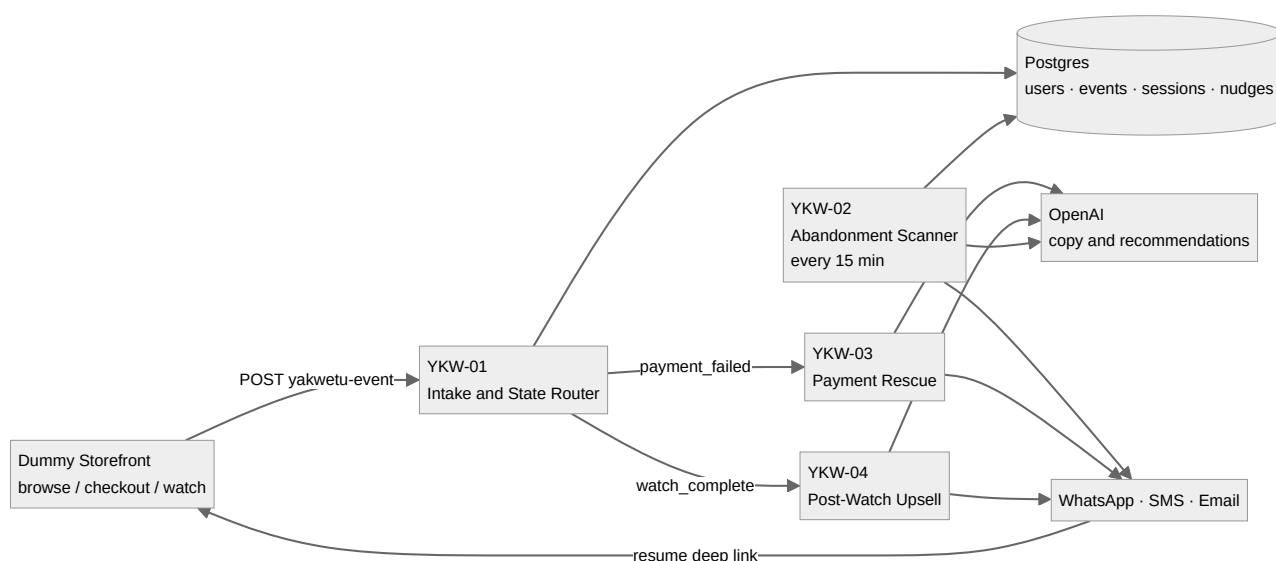


Figure 1 End-to-end architecture

Table 2 Component responsibilities

Component	Technology	Responsibility
Storefront simulator	Static HTML/JS (3 pages)	Browse grid, M-Pesa-style checkout with success/failure simulation, fake player; emits the production event contract
Intake & router	n8n workflow YKW-01	Validate + normalize events, upsert users, log events, maintain session state, route to scenarios, attribute recoveries
State layer	Postgres (schema . sql)	Users, raw events, per-session state machine, nudge log with conversion attribution, drop-off classifications
Abandonment scanner	n8n schedule trigger, YKW-02	Finds idle unpurchased sessions, enforces quiet hours + caps, triggers personalized nudges

Copy recommendations	& OpenAI gpt-4o-mini	Personalized nudge text, failure-specific troubleshooting, bundle curation — JSON-constrained output
Channels	Twilio WhatsApp, Africa's Talking SMS, SMTP email	Delivery with cascade fallback; sender ID YAKWETU
Analytics	Postgres queries + Google Sheets	Funnel, nudge ROI by scenario/channel, top drop-off causes

4. Event Contract & Data Model

All client-side tracking reduces to six event types over one webhook. The dummy pages already emit exactly this shape; the production app would emit the same from its real player and Daraja callbacks.

```
{
  "event_type": "browse | signup | checkout_start | payment_failed | payment_success | watch_complete",
  "user_id": "u_1024", "name": "Wanjiku Mwangi",
  "phone": "+2547XXXXXXX", "email": "viewer@example.com",
  "session_id": "sess_abc123",
  "movie_id": "nairobi-nights", "movie_title": "Nairobi Nights",
  "genre": "Crime Drama", "price_kes": 150,
  "failure_reason": "Insufficient balance" // payment_failed only
}
```

The session table is a small state machine: `browsing` → `checkout` → `failed` → `purchased`, with `recovered`, `nudge_count`, and `last_nudge_at` fields that drive every targeting decision. A purchase inside 72 h of a nudge flips `nudges.converted = true` — that single join is the entire attribution model.

5. Scenario A — Browse & Cart Abandonment Recovery (YKW-02)

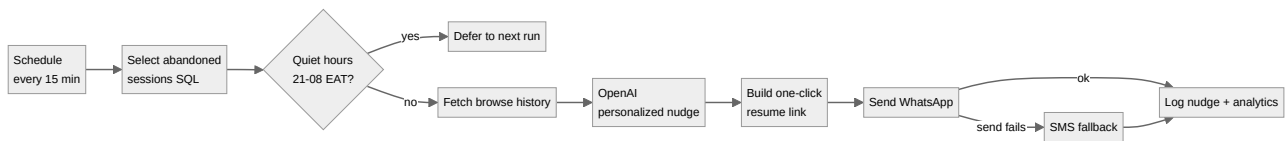


Figure 2 Abandonment recovery flow

Table 3 Targeting rules (all in one SQL query)

Rule	Value	Reason
Idle threshold	45 minutes	Long enough to be abandonment, short enough to still remember the title
Expiry	72 hours	Older intent is cold; AI copy stops converting
Frequency cap	2 nudges per session, ≥20 h apart	Protects brand trust; third nudge never pays for itself
Quiet hours	21:00–08:00 EAT deferred	Night nudges read as spam in Kenya; deferral, not dropping, preserves the lead
Suppression	<code>recovered = false</code> required	Anyone who bought since is excluded automatically

The AI step receives the viewer's five most recent browsed titles and returns strict JSON (`message`, `recommended_title`) with a 300-character cap and a tone brief ("friendly Nairobi energy, light Sheng allowed, never desperate"). A code node then appends the resume deep link — `checkout.html?resume={session_id}` — which re-opens checkout with the phone prefilled and the exact movie restored. One tap from WhatsApp to an STK prompt: that is the entire friction budget.

6. Scenario B — Payment Failure Rescue (YKW-03)

Payment failure is the highest-intent moment that exists — the customer already said yes. The workflow classifies the Daraja-style failure reason and tailors the fix:

Table 4 Failure triage matrix

Failure class	Detection keywords	Message strategy
Insufficient funds	insufficient, balance	Reassure no money lost; offer the cheaper 24-hour rental option
Wrong PIN	pin, authorisation	Explain the phone prompt, send a fresh STK push
Timeout cancelled	/ timeout, cancelled	One-tap fresh prompt; note network congestion
Till limit exceeded	limit	Offer card payment alternative

The rescue WhatsApp fires within seconds of the failure — while the customer is still on the checkout screen. A Wait node then parks the execution for 2 hours; on wake it re-reads the session, and only if still unrecovered sends an SMS with a time-boxed incentive code

(PONA10, 10% off, tonight only). Every failure is also written to a `dropoffs` table, so "why do payments fail" becomes a weekly report rather than a guess.

7. Scenario C — Post-Watch Personalized Upsell (YKW-04)

The credits are the emotional peak of the product — the best possible moment to sell the next story. One hour after `watch_complete`, the workflow pulls the viewer's recent watch history, and the AI curates a 2–3 title bundle in the same genre at roughly 20% off, with a 48-hour expiry. The bundle goes out as a styled HTML email (dark palette, gold accents, single CTA). If no purchase lands within 24 hours, a short WhatsApp "last call" closes the sequence. The second touch is conditional on the first not converting — the engine never argues with a customer who already bought.

8. Making It Seamless — Creative Details

1. **Zero-typing recovery.** The resume link carries the session; checkout re-opens with movie, price, and M-Pesa number prefilled, and can auto-fire the STK push on load. From nudge to payment prompt: one tap, zero keystrokes.
2. **Smart-send timing.** Because every event is timestamped, a later iteration can nudge each user at their personal median-active hour instead of a fixed delay — a one-line SQL change.
3. **Failure empathy, not error codes.** The AI is explicitly instructed to lead with "no money was lost / reversals are automatic" — the single biggest anxiety in Kenyan mobile-money payments.
4. **Escalating value, not escalating volume.** Touch 1 is a reminder; touch 2 adds an incentive; there is no touch 3. Pressure comes from the offer improving, not from pestering.
5. **Attribution built-in.** Recovery is attributed to the exact nudge and channel, so the demo can close with "WhatsApp recovered X% of abandoned checkouts" — the number an investor actually asks for.
6. **Offline-tolerant demo.** The storefront queues events locally if n8n is unreachable, so a venue Wi-Fi failure never kills the presentation.
7. **Reply-to-resume (roadmap).** Africa's Talking two-way SMS can map "Reply 1" to a fresh STK push — no smartphone or data bundle required, which matters for the mass market.

9. Setup & Demo Script

9.1 Setup (30–45 minutes)

1. Run `schema.sql` on any Postgres (the n8n database itself is fine).
2. Import all four workflow JSONs into n8n (Workflows → Import from File).
3. Create credentials and re-select them on flagged nodes: Postgres, OpenAI, Twilio, SMTP, Google Sheets (optional), and the `AT_API_KEY` environment variable for Africa's Talking sandbox.
4. In YKW-01, point the two Execute Workflow nodes at YKW-03 and YKW-04 (IDs are blank after import).
5. Activate YKW-01 and YKW-02. Expose local n8n via `ngrok` or `localtunnel`, and paste the base URL into the settings bar on any storefront page.

9.2 Five-minute demo

1. Browse 2–3 movies on the storefront — events appear in the `events` table.
2. Buy one, enter a phone, and click **Simulate: Insufficient funds** — a tailored rescue WhatsApp arrives within seconds.
3. Tap the resume link in the message; checkout re-opens prefilled. Click **Simulate: Success** — the session flips to recovered and the nudge is attributed.
4. On the watch page, click **Simulate: finish watching** — the upsell email with a discounted bundle arrives after the wait.
5. For Scenario A on stage, temporarily set the idle threshold to 1 minute and the schedule to every minute.
6. Close with the funnel queries at the bottom of `schema.sql` — state counts, recovery rate by channel, top failure causes.

10. Path to Production

Table 5 Prototype → production swaps

Prototype	Production	Effort
Simulated buttons	STK M-Pesa Daraja STK Push + callbacks (payload shape already mirrors ResultCode / ResultDesc)	Low

10. Path to Production

Plain resume session link	Signed single-use token (HMAC of session + expiry)	Low
Twilio WhatsApp sandbox	WhatsApp Business Cloud API with approved templates	Medium approval (template approval)
Wait-node holds	Queue-based re-checks beyond ~10k concurrent waits	Medium
Fixed 45-min idle rule	Per-user smart-send hour from event timestamps	Low
SQL + Sheets analytics	Metabase/Superset dashboard on the same tables	Low

Deliverables accompanying this document: four importable n8n workflow JSONs, `schema.sql`, a README with credentials checklist, and the three-page dummy storefront (browse, checkout, watch) with built-in event tracking and offline queueing.